

Time-Space Tradeoffs of Truncation with Preprocessing

Research Paper Notes

1 Problem and Objective

Consider a random function

$$f : [N] \rightarrow [N].$$

The attacker receives a challenge y and must find its preimage x :

$$f(x) = y$$

The attacker is allowed to preprocess f and store S bits of information. During the actual attack, it can make at most T queries to f .

If the attacker successfully inverts an ϵ fraction of possible challenges, the standard time-space tradeoff is

$$ST \in \Omega(\epsilon N)$$

The main objective - Does truncating the output make this tradeoff significantly easier to break?

2 Truncation

Suppose only outputs satisfying a particular condition are considered. For example, the first Δ output bits may have to be zero.

For a random output, the probability of satisfying this condition is

$$\delta = 2^{-\Delta}$$

Therefore, the sparse/truncated range has approximately

$$\delta N$$

possible outputs.

Thus the attacker only needs to invert challenges from this smaller range.

If the attacker can simply store the preimage of every truncated output, it can answer a challenge almost immediately. However, this requires approximately

$$S \approx \delta N \log N$$

bits of storage.

Hence, truncation can create a problem as there is lesser outputs.

3 Meaning of ϵ and Successful Inversion

In one actual attack, the attacker receives only one challenge y .

It succeeds if it outputs x satisfying

$$f(x) = y$$

For example, if

$$f(123) = 57$$

and the challenge is $y = 57$, outputting $x = 123$ is a success.

If $f(500) \neq 57$, then outputting 500 is a failure.

The value ϵ describes the success probability over all possible challenges.

Thus,

$$\epsilon N$$

is approximately the number of possible challenges on which the attacker succeeds.

In the truncated case, the number of possible challenges is δN , so the number of successful truncated challenges is approximately

$$\epsilon \delta N$$

4 Compression Argument

The main proof uses a compression argument.

Suppose the attacker successfully solves y by finding x such that

$$f(x) = y.$$

Then the pair

$$(x, y)$$

gives us one function-table entry for free.

Therefore, many successful inversions allow us to describe the random function using fewer bits. But a random function should be essentially incompressible. Hence, the attacker must have already stored enough information in its preprocessing storage S .

5 Spoiled Challenges and $\epsilon N/T$

Suppose there are approximately ϵN successful challenges available.

For each attack, the attacker can make at most T queries.

Some queried values may correspond to other possible challenges. These challenges are then spoiled and cannot be safely used again in the compression argument. Thus one successful inversion can spoil up to T challenges.

Therefore the approximate number of usable successful inversions is

$$\frac{\epsilon N}{T}$$

This gives

$$S \gtrsim \frac{\epsilon N}{T}.$$

$$ST \in \Omega(\epsilon N)$$

6 Truncated Case and T_g

In the truncated case, only δN outputs are relevant.

Let T_g denote the number of queries whose outputs land in the sparse range.

For a random query, the probability of landing in the sparse range is approximately δ . Therefore,

$$\mathbb{E}[T_g] \approx \delta T.$$

The proof considers two cases.

7 Case 1: $T_g \leq 12\delta T$

If

$$T_g \leq 12\delta T$$

the attacker is not making an unusually large number of queries into the sparse range.

Therefore, query spoiling is limited and the compression argument essentially gives the usual tradeoff:

$$ST \in \Omega(\epsilon N)$$

8 Case 2: $T_g > 12\delta T$

If

$$T_g > 12\delta T$$

the attacker is querying the sparse range unusually often.

This seems helpful to the attacker, but it creates another compression opportunity. A normal output requires approximately $\log N$ bits to describe. A sparse output only requires $\log(\delta N)$ bits.

Therefore the saving is

$$\log N - \log(\delta N) = \log(1/\delta)$$

bits per sparse output.

Thus, many sparse outputs allow the function to be compressed significantly. Since the random function should be incompressible, the attacker must already have significant preprocessing storage.

The resulting bound is

$$S \in \Omega(\delta\epsilon N)$$

9 Main Result

Combining the two cases gives the main result:

$$S \in \Omega(\delta\epsilon N) \quad \text{OR} \quad ST \in \Omega(\epsilon N)$$

Therefore, truncation does not simply eliminate the fundamental space-time tradeoff.

The attacker must pay either through:

large preprocessing storage

or

large space-time product.